BÁO CÁO THỰC HÀNH LAB A11

Dialog, Toast & Menu — Nâng cấp ứng dụng “Thực đơn quán ăn”

Options menu · Context menu · AlertDialog · Hộp thoại tự thiết kế · Snackbar Hoàn tác · NC1 + NC2 + NC3

Họ và tên: Nguyễn Việt Quang
MSSV: 231A290134
Lớp: CNTT – LTDD

Android Studio Quail 3 | 2026.1.3
Tên AVD: Pixel 6 API 34 (điện thoại, 1080 × 2400)
Phiên bản Android: Android 14 — Kiến trúc: x86_64
Link GitHub: https://github.com/nvquangggit-sys/A11_231A290134

Checkpoint

Checkpoint — sản phẩm bắt buộc

Chạy app: thanh tiêu đề có biểu tượng ＋, kính lúp và nút ba chấm; giữ lâu một dòng ra context menu đúng tên món; chọn Xóa thì có hộp thoại hỏi lại và nêu rõ tên món; bấm Xóa rồi bấm HOÀN TÁC thì món trở về đúng vị trí cũ; mở dialog Sửa, xóa trắng ô tên rồi bấm Lưu thì hộp thoại không đóng và báo lỗi ngay tại ô tên.

Bài làm là bản nâng cấp trực tiếp của project Lab A9: giữ nguyên lớp dữ liệu MonAn, adapter tự viết với ViewHolder và ListView, sau đó thay ô nhập liệu ở đầu màn hình bằng options menu + hộp thoại, thay listener giữ lâu cũ bằng context menu, và bổ sung ba lớp hộp thoại cùng Snackbar Hoàn tác. Toàn bộ ảnh minh chứng trong báo cáo được chụp trên emulator Pixel 6 API 34 (1080 × 2400) từ chính bản build đang chạy, không qua chỉnh sửa.

Phần 1. Ba cách nói chuyện với người dùng

Cùng một việc "báo cho người dùng biết", Android có ba công cụ với ba mức can thiệp rất khác nhau. Chọn sai công cụ không làm app crash nhưng làm trải nghiệm nặng nề: hỏi xác nhận cho một việc vô hại, hoặc tệ hơn là xóa dữ liệu mà không cho người ta cơ hội cứu.

	Toast	Snackbar	AlertDialog
Vị trí	Nổi gần giữa/cuối màn hình	Trượt lên từ cạnh dưới	Giữa màn hình, làm mờ nền
Chặn thao tác?	Không	Không	Có — phải trả lời mới dùng tiếp
Có nút bấm?	Không	Một nút (ví dụ HOÀN TÁC)	1–3 nút
Dùng khi	Báo việc đã xong, không quan trọng	Báo việc đã xong và cho cơ hội sửa	Hỏi trước khi làm việc nguy hiểm

Quy tắc chọn của bài:

- Việc không thể hoàn tác (xóa vĩnh viễn, gửi, thanh toán) → hỏi bằng AlertDialog.
- Việc hoàn tác được → cứ làm rồi báo bằng Snackbar kèm nút Hoàn tác.
- Việc nhỏ, không cần hành động gì thêm → Toast.

Áp vào bài: sắp xếp xong → Toast "Đã sắp xếp theo giá tăng dần"; xóa một món → AlertDialog xác nhận nêu rõ tên món, sau đó Snackbar "Đã xóa …" kèm HOÀN TÁC. Có ý kiến cho rằng đã có Hoàn tác thì không cần hỏi trước; bài vẫn giữ cả hai lớp vì (1) đề yêu cầu rõ hộp thoại xác nhận, và (2) xóa món là thao tác phá hủy dữ liệu người dùng vừa nhập, còn Hoàn tác chỉ là lưới an toàn cho cú bấm nhầm thứ hai. Gmail cũng làm vậy với thư đã xóa, nhưng vẫn hỏi khi xóa tài khoản vĩnh viễn.

Phần 2. Nâng cấp project A9 & options menu

2.1. Những gì đã đổi so với A9

Yêu cầu của đề là nâng cấp, không tạo project mới. Vì vậy bài chép nguyên thư mục A9 (bỏ .git, build, .gradle, .idea), đổi tên gói bằng Refactor → Rename, sửa app_name rồi chạy thử để chắc app vẫn hoạt động trước khi viết thêm code. Phần hiển thị danh sách giữ nguyên từ A9 — lớp MonAn, MonAnAdapter với ViewHolder và cả hiệu ứng tô nền vàng cho món giá trên 45.000 đ; đó là lý do dòng "Bún chả Hà Nội" có nền nhạt trong nhiều ảnh, không phải dòng đang được chọn.

Hạng mục	Trước (Lab A9)	Sau (Lab A11)
Tên project	A9_231A290134	A11_231A290134
Package / applicationId	vn.edu.vhu.ltdd.a9listview	vn.edu.vhu.ltdd.a11dialog
app_name	A9_231A290134	A11_231A290134
Thêm món	Hai ô nhập + nút Thêm trên màn hình	Mục ＋ trên menu → hộp thoại tự thiết kế

Hạng mục	Trước (Lab A9)	Sau (Lab A11)
Giữ lâu	setOnItemLongClickListener xóa ngay	registerForContextMenu → Sửa món / Xóa
Xóa	Xóa ngay, chỉ báo Toast	AlertDialog xác nhận + Snackbar Hoàn tác
Theme	Theme.A9ListView (NoActionBar)	Theme.A11Dialog + MaterialToolbar

2.2. res/menu/menu_main.xml

Menu của cả màn hình nằm trong XML. Hai mục dùng nhiều nhất (Thêm, Tìm kiếm) đặt showAsAction="always" để hiện thành biểu tượng; ba mục còn lại để "never" cho thanh tiêu đề gọn.

```
<!-- res/menu/menu_main.xml -->
<item android:id="@+id/action_them"
    android:icon="@android:drawable/ic_input_add"
    android:title="@string/them_mon" app:showAsAction="always" />

<!-- NC1: SearchView thu gọn thành kính lúp, chạm để mở rộng -->
<item android:id="@+id/action_tim_kiem"
    android:icon="@android:drawable/ic_menu_search"
    android:title="@string/hint_tim"
    app:actionViewClass="androidx.appcompat.widget.SearchView"
    app:showAsAction="always|collapseActionView" />

<item android:id="@+id/action_sap_xep_ten" android:title="@string/sap_xep_ten"
    app:showAsAction="never" />
<item android:id="@+id/action_sap_xep_gia" android:title="@string/sap_xep_gia"
    app:showAsAction="never" />
<item android:id="@+id/action_loc_gia" android:title="@string/loc_gia"
    app:showAsAction="never" />
<item android:id="@+id/action_xoa_het" android:title="@string/xoa_het"
    app:showAsAction="never" />
```

showAsAction	Hiển thị thế nào	Dùng cho mục nào trong bài
always	Luôn là biểu tượng trên thanh tiêu đề	Thêm món, tìm kiếm
ifRoom	Là biểu tượng nếu còn chỗ, ngược lại vào menu ba chấm	Không dùng
never	Luôn nằm trong menu ba chấm	Sắp xếp theo tên/giá, lọc giá, xóa tất cả
withText	Kèm chữ bên cạnh biểu tượng	Không dùng

2.3. onCreateOptionsMenu và onOptionsItemSelected

```
@Override
public boolean onCreateOptionsMenu(Menu menu) {
    getMenuInflater().inflate(R.menu.menu_main, menu);
    return true;    // true = hiện menu; trả false thì menu không xuất hiện
}

@Override
public boolean onOptionsItemSelected(@NonNull MenuItem item) {
    int id = item.getItemId();
    // Dùng if-else, KHÔNG dùng switch: từ AGP 8, R.id không còn là hằng số biên dịch.
    if (id == R.id.action_them) {
        hienDialogThemSua(null);
        return true;
    } else if (id == R.id.action_sap_xep_ten) {
        sapXepTheoTen();
        return true;
    } else if (id == R.id.action_sap_xep_gia) {
        sapXepTheoGia();
        return true;
    } else if (id == R.id.action_loc_gia) {
        hienDialogChonMucGia();          // NC2
        return true;
    } else if (id == R.id.action_xoa_het) {
        xacNhanXoaHet();
        return true;
    }
    return super.onOptionsItemSelected(item);
}
```

Ba chi tiết quyết định menu có chạy hay không:

- getMenuInflater().inflate(...) biến file XML thành đối tượng Menu thật; thiếu dòng này thì không có gì hiện.
- return true trong onOptionsItemSelected nghĩa là “tôi đã xử lý, dừng ở đây”; trả false thì sự kiện rơi xuống Activity và có thể mở thêm màn hình khác.
- Dòng cuối return super.onOptionsItemSelected(item) để các mục không xử lý (ví dụ nút Back trên thanh tiêu đề) vẫn chạy bình thường. Bỏ dòng này là lỗi tinh vi, rất khó tìm.

Vì sao phải thêm MaterialToolbar?

Theme Material3.Light.NoActionBar không có ActionBar, nên options menu không có chỗ để hiện — app chạy không lỗi mà cũng không có menu. Bài thêm MaterialToolbar vào activity_main.xml rồi gọi setSupportActionBar(toolbar) trong onCreate; từ đó getMenuInflater mới có ActionBar để gắn menu.

Phần 3. Context menu — phần khó nhất của bài

Context menu là menu của một dòng, không phải của cả màn hình. Chỉ cần một dòng đăng ký là ListView tự bắt sự kiện giữ lâu:

```
// Đăng ký MỘT lần trong onCreate
registerForContextMenu(lvMonAn);

// KHÔNG đăng ký setOnItemLongClickListener nữa:
// listener đó nuốt sự kiện giữ lâu và context menu sẽ không bao giờ hiện.
```

Khi menu mở, Android gọi onCreateContextMenu. Việc quan trọng nhất ở đây là đặt tiêu đề menu bằng đúng tên món — người dùng cần biết mình đang thao tác trên dòng nào trước khi bấm Xóa:

```
@Override
public void onCreateContextMenu(ContextMenu menu, View v,
                                ContextMenu.ContextMenuInfo menuInfo) {
    super.onCreateContextMenu(menu, v, menuInfo);
    if (v.getId() == R.id.lvMonAn) {
        AdapterView.AdapterContextMenuInfo info =
                (AdapterView.AdapterContextMenuInfo) menuInfo;
        MonAn mon = hienThi.get(info.position);
        menu.setHeaderTitle(mon.getTen());   // tiêu đề = đúng tên món được giữ lâu
        getMenuInflater().inflate(R.menu.menu_context, menu);
    }
}
```

Lúc người dùng chọn một mục, vị trí dòng không nằm trong MenuItem mà nằm trong MenuInfo mà Android gửi kèm:

```
@Override
public boolean onContextItemSelected(@NonNull MenuItem item) {
    AdapterView.AdapterContextMenuInfo info =
            (AdapterView.AdapterContextMenuInfo) item.getMenuInfo();
    if (info == null) return super.onContextItemSelected(item);

    MonAn mon = hienThi.get(info.position);      // đối tượng đang hiển thị
    int id = item.getItemId();
    if (id == R.id.ctx_sua) {
        hienDialogThemSua(mon);
        return true;
    } else if (id == R.id.ctx_xoa) {
        xacNhanXoaMot(mon);
        return true;
    }
    return super.onContextItemSelected(item);
}
```

Bài có hai danh sách: danhSach là dữ liệu gốc, hienThi là danh sách sau khi lọc NC1/NC2. Vì vậy info.position là chỉ số trong hienThi, không phải trong danhSach. Cách làm an toàn là lấy đối tượng MonAn trước, rồi xóa theo danhSach.indexOf(mon) — nhờ đó xóa đúng món kể cả khi đang lọc.

```
// trong xoaCoHoanTac(MonAn mon)
final int viTri = danhSach.indexOf(mon);   // vị trí thật trong dữ liệu gốc
if (viTri < 0) return;
final MonAn daXoa = danhSach.remove(viTri);
```

Lỗi gần như chắc chắn xảy ra khi nâng cấp từ A9

A9 có lvMonAn.setOnItemLongClickListener(...) để xóa ngay. Listener đó nuốt sự kiện giữ lâu nên context menu không bao giờ hiện, dù registerForContextMenu đã gọi đúng. Bài đã xóa hẳn listener cũ; nhờ vậy giữ lâu chỉ mở menu, không xóa dữ liệu.

Phần 4. AlertDialog xác nhận trước khi xóa

Hộp thoại xác nhận là chỗ dễ mất điểm nhất vì đề chấm cả nội dung lời hỏi. Lời hỏi phải nêu rõ đối tượng sẽ bị tác động và nhãn nút phải là động từ của việc sắp làm:

```
private void xacNhanXoaMot(MonAn mon) {
    new MaterialAlertDialogBuilder(this)          // NC3
            .setTitle(R.string.xac_nhan_xoa)
            .setMessage(getString(R.string.hoi_xoa, mon.getTen()))
            .setPositiveButton(R.string.xoa, (d, w) -> xoaCoHoanTac(mon))
            .setNegativeButton(R.string.huy, null)   // null = chỉ đóng hộp thoại
            .show();
}

// res/values/strings.xml
<string name="hoi_xoa">Bạn có chắc muốn xóa \"%1$s\" khỏi thực đơn?</string>
```

Kết quả trên máy: “Bạn có chắc muốn xóa "Cơm tấm sườn" khỏi thực đơn?” với hai nút Hủy / Xóa. Nếu chỉ ghi “Bạn có chắc không?” thì người dùng không biết mình đang xóa món nào — đề ghi rõ trường hợp này bị trừ nửa điểm. Việc xóa chỉ xảy ra trong listener của nút Xóa, nên bấm Hủy hoặc chạm ra ngoài đều không làm gì.

Xóa tất cả cũng dùng cùng khuôn, nhưng lời hỏi nêu số lượng: “Xóa toàn bộ 5 món trong thực đơn?”. Nếu danh sách đã rỗng thì chỉ hiện Toast “Thực đơn trống — dùng nút + để thêm món” thay vì mở hộp thoại vô nghĩa.

Phần 5. Hộp thoại tự thiết kế (Thêm / Sửa món)

5.1. Layout nội dung

Hộp thoại dùng lại đúng khuôn ba ô nhập của đề, đặt trong một layout riêng rồi nhét vào bằng setView. Nhờ vậy cùng một hộp thoại phục vụ cả Thêm và Sửa; chỉ khác dữ liệu điền sẵn và tiêu đề.

```
<!-- res/layout/dialog_mon_an.xml -->
<LinearLayout android:orientation="vertical"
    android:paddingStart="24dp" android:paddingEnd="24dp"
    android:paddingTop="12dp" android:paddingBottom="4dp">
    <EditText android:id="@+id/edtTen"  android:hint="@string/hint_ten"
        android:inputType="text" android:importantForAutofill="no" />
    <EditText android:id="@+id/edtMoTa" android:hint="@string/hint_mo_ta"
        android:inputType="text" android:importantForAutofill="no" />
    <EditText android:id="@+id/edtGia"  android:hint="@string/hint_gia"
        android:inputType="number" android:importantForAutofill="no" />
</LinearLayout>
```

5.2. Mẹo quan trọng nhất: gán listener SAU show()

Listener truyền qua setPositiveButton luôn bị Android đóng hộp thoại sau khi chạy xong. Nếu dữ liệu chưa hợp lệ, người dùng sẽ mất hết những gì vừa gõ. Cách xử lý là truyền null lúc tạo, gọi show(), rồi gán lại OnClickListener cho nút Lưu — từ đó code tự quyết định lúc nào gọi dismiss():

```
AlertDialog dialog = new MaterialAlertDialogBuilder(this)
        .setTitle(themMoi ? R.string.them_mon : R.string.sua_mon)
        .setView(v)
        .setPositiveButton(R.string.luu, null)    // gán null rồi ghi đè ở dưới
        .setNegativeButton(R.string.huy, null)
        .create();

dialog.show();

dialog.getButton(AlertDialog.BUTTON_POSITIVE).setOnClickListener(btn -> {
    String ten = edtTen.getText().toString().trim();
    String moTa = edtMoTa.getText().toString().trim();
    String giaStr = edtGia.getText().toString().trim();

    if (ten.isEmpty()) {
        edtTen.setError(getString(R.string.err_ten));
        edtTen.requestFocus();
        return;            // KHÔNG đóng hộp thoại
    }
    int gia;
    try {
        gia = Integer.parseInt(giaStr);
    } catch (NumberFormatException e) {
        edtGia.setError(getString(R.string.err_gia));
        return;
    }
    if (gia <= 0) {
        edtGia.setError(getString(R.string.err_gia_duong));
        return;
    }
    // … lưu dữ liệu …
    dialog.dismiss();      // tự tay đóng khi mọi thứ đã hợp lệ
});
```

Trường hợp	Thông báo tại ô	Hành vi của hộp thoại
Tên để trắng	Chưa nhập tên món	Không đóng, focus lại ô tên, dữ liệu còn nguyên
Giá không phải số	Giá phải là số	Không đóng, focus lại ô giá
Giá ≤ 0	Giá phải lớn hơn 0	Không đóng, focus lại ô giá
Hợp lệ	—	Lưu vào danh sách, đóng hộp thoại, hiện Toast

Vì sao phải là show() rồi mới gán? Vì getButton(...) chỉ trả về đối tượng Button sau khi hộp thoại đã được dựng; gọi trước show() sẽ ném NullPointerException. Còn nếu gán listener ngay trong setPositiveButton thì AlertDialog tự gọi dismiss() sau callback, không có cách nào chặn — đó chính là hiện tượng "gõ thiếu một ô là mất sạch dữ liệu".

Phần 6. Snackbar và chức năng Hoàn tác

Xóa xong, món được giữ lại trong biến và cho người dùng vài giây để cứu. Điểm mấu chốt là phải giữ cả đối tượng lẫn vị trí, nếu không món sẽ quay về cuối danh sách:

```
private void xoaCoHoanTac(MonAn mon) {
    final int viTri = danhSach.indexOf(mon);
    if (viTri < 0) return;

    final MonAn daXoa = danhSach.remove(viTri);
    locVaCapNhat();
    Log.d(TAG, "Đã xóa " + daXoa.getTen() + " tại vị trí " + viTri);

    Snackbar.make(lvMonAn, getString(R.string.da_xoa, daXoa.getTen()),
                  Snackbar.LENGTH_LONG)
            .setAction(R.string.hoan_tac, v -> {
                danhSach.add(viTri, daXoa);      // trả về ĐÚNG vị trí cũ
                locVaCapNhat();
                int hien = hienThi.indexOf(daXoa);
                if (hien >= 0) lvMonAn.smoothScrollToPosition(hien);
                Log.d(TAG, "Hoàn tác: " + daXoa.getTen() + " về vị trí " + viTri);
            })
            .show();
}
```

- Tham số đầu của Snackbar.make là một View bất kỳ trong màn hình; Snackbar tự tìm khung chứa phù hợp để bám vào. Bài dùng lvMonAn nên Snackbar nổi ở đáy màn hình, không che thanh tiêu đề.
- LENGTH_LONG đủ lâu để người dùng kịp đọc tên món và bấm HOÀN TÁC.
- Sau khi hoàn tác, danh sách được lọc lại và cuộn tới vị trí của món — kể cả khi đang lọc theo từ khóa.
- Xóa tất cả dùng cách khác: sao lưu cả danhSach vào banSaoLuu rồi addAll khi bấm Hoàn tác, vì không có một vị trí duy nhất để chèn lại.

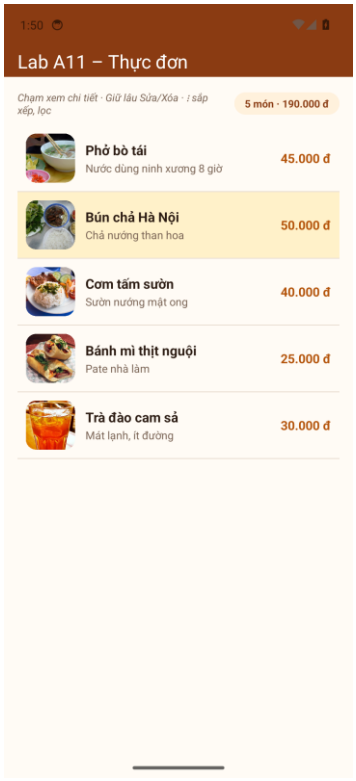
Vì sao add(viTri, daXoa) chứ không phải add(daXoa)?

add(daXoa) luôn chèn vào cuối danh sách. Người dùng xóa món thứ ba rồi bấm Hoàn tác sẽ thấy nó nhảy xuống dưới cùng — dữ liệu không mất nhưng thứ tự thực đơn đã đổi, và nếu đang sắp xếp theo giá thì danh sách sai thứ tự. add(viTri, daXoa) mới thật sự là hoàn tác.

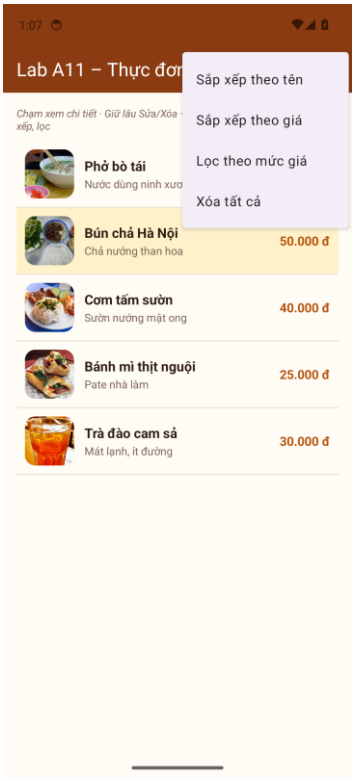
Phần 7. Kiểm thử (8 kịch bản của đề)

Thiết bị kiểm thử: Pixel 6 API 34, Android 14, 1080 × 2400, chế độ sáng. Mỗi kịch bản được chạy lại từ trạng thái sạch (am force-stop + mở mới, chờ app hiện rồi chạm vùng trống để ListView nhận focus) trước khi chụp, trừ các bước cố ý nối tiếp nhau (ví dụ Hủy rồi Xóa trong cùng một phiên).

#	Thao tác	Kết quả đúng phải thấy	Kết quả thực tế & minh chứng
1	Mở app, nhìn thanh tiêu đề	Có biểu tượng ＋ và nút ba chấm; danh sách 5 món mẫu	Thấy ＋, kính lúp, ⋮; chip “5 món · 190.000 đ” — Hình 1a
2	Mở menu ba chấm	Đủ Sắp xếp theo tên / theo giá / Xóa tất cả	Có thêm “Lọc theo mức giá” (NC2): tổng 4 mục — Hình 1b
3	Chọn Sắp xếp theo giá	Danh sách xếp tăng dần, Toast báo đã sắp xếp	25 → 30 → 40 → 45 → 50 (nghìn); Toast “Đã sắp xếp theo giá tăng dần” — Hình 1c
4	Giữ lâu dòng thứ 3 (Cơm tấm sườn)	Context menu hiện, tiêu đề là tên đúng món đó	Menu có tiêu đề “Cơm tấm sườn”, hai mục Sửa món / Xóa — Hình 1d
5	Chọn Xóa trong context menu	Hộp thoại hỏi lại, nêu đúng tên món	“Bạn có chắc muốn xóa "Cơm tấm sườn" khỏi thực đơn?” — Hình 2a
6	Bấm Hủy trong hộp thoại	Không xóa gì, danh sách giữ nguyên	Vẫn đủ 5 món · 190.000 đ — Hình 2b
7	Bấm Xóa rồi bấm HOÀN TÁC	Món trở lại đúng vị trí cũ, không phải cuối danh sách	Còn 4 món + Snackbar “Đã xóa Cơm tấm sườn” (Hình 2c); Hoàn tác đưa Cơm tấm về vị trí thứ 3 (Hình 2d)
8	Mở dialog Sửa, xóa trắng ô tên, bấm Lưu	Báo lỗi tại ô tên, hộp thoại không đóng	Ô tên báo đỏ “Chưa nhập tên món”, hộp thoại và dữ liệu còn nguyên — Hình 3a, 3b



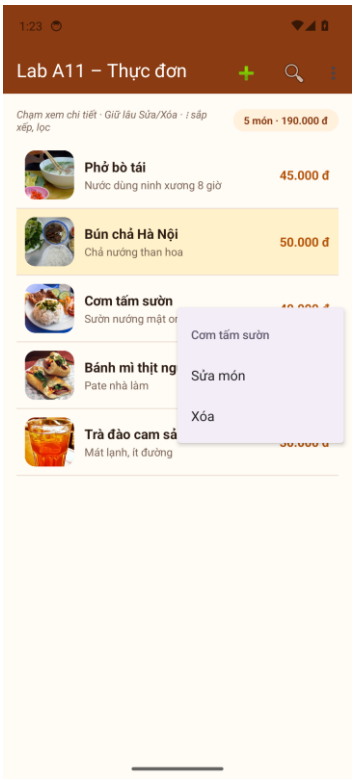
(a) KB1 — +, kính lúp, ⋮ và chip thống kê



(b) KB2 — menu ba chấm đủ 4 mục

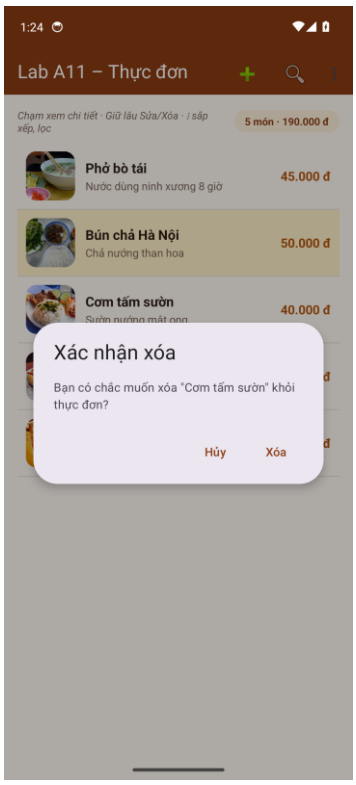


(c) KB3 — xếp theo giá + Toast

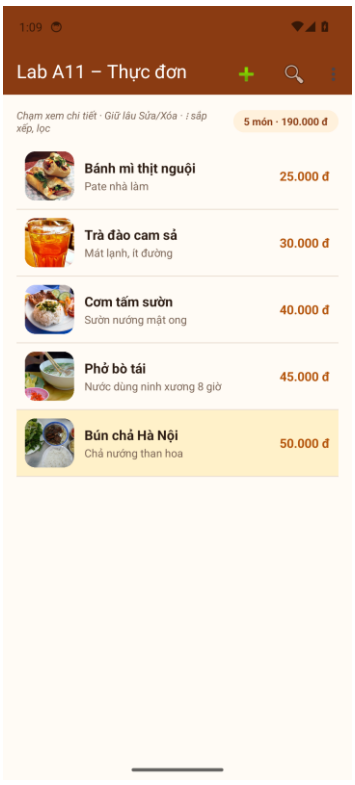


(d) KB4 – context menu đúng tên món

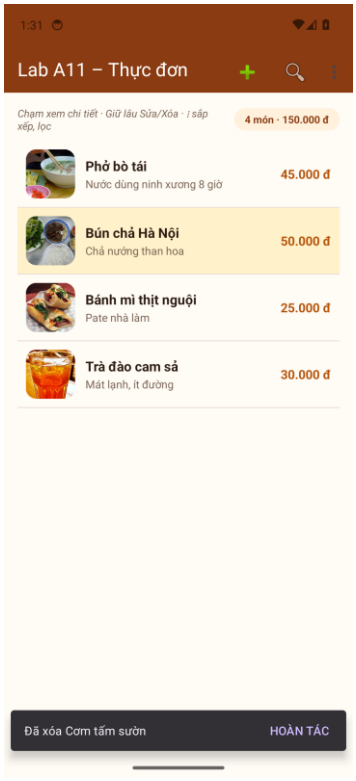
Hình 1. Minh chứng kịch bản 1–4 (Pixel 6 API 34, chụp từ app đang chạy).



(a) KB5 — lời hỏi nêu rõ tên món



(b) KB6 — Hủy: danh sách vẫn 5 món

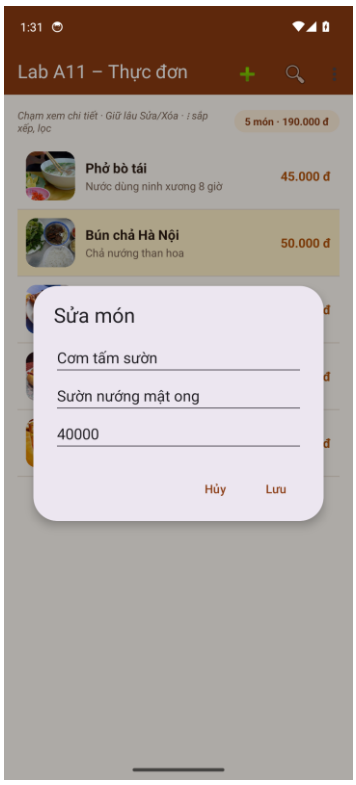


(c) KB7 — còn 4 món + Snackbar Hoàn tác

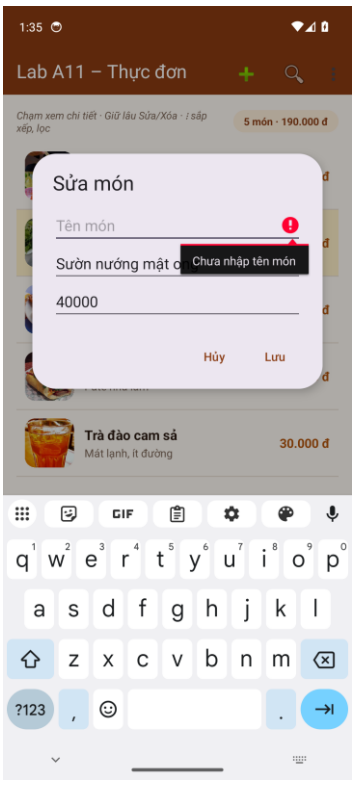


(d) KB7 — Hoàn tác: Cơm tấm về vị trí thứ 3

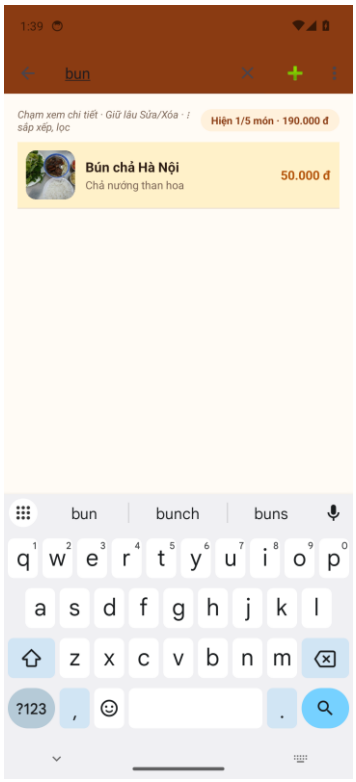
Hình 2. Minh chứng kịch bản 5–7.



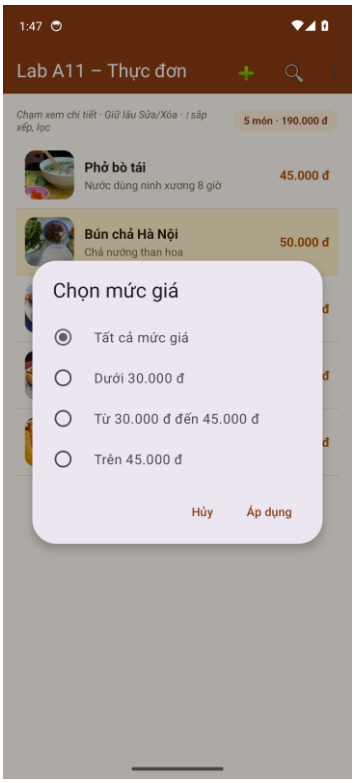
(a) KB8 — dialog Sửa điền sẵn dữ liệu



(b) KB8 — lỗi tại ô tên, hộp thoại không đóng



(c) NC1 — gõ "bun" còn 1/5 món



(d) NC2 — hộp thoại chọn mức giá

Hình 3. Minh chứng kịch bản 8 và hai bài nâng cao NC1, NC2.

Logcat thật của TAG A11_231A290134 (trích docs/logs/logcat_a11.txt) xác nhận vị trí dòng được lấy đúng từ AdapterContextMenuInfo và hoàn tác trả về đúng chỗ:

```
D A11_231A290134: Context menu Sửa: Cơm tấm sườn (vị trí 2)
D A11_231A290134: Context menu Xóa: Cơm tấm sườn (vị trí 2)
D A11_231A290134: Đã xóa Cơm tấm sườn tại vị trí 2
D A11_231A290134: Hoàn tác: Cơm tấm sườn về vị trí 2
D A11_231A290134: Đã sắp xếp theo giá tăng dần
```

7.1. Ba câu hỏi phân tích

1. Ở kịch bản 4, thử giữ lâu dòng thứ 3 rồi chọn Sửa. Nếu lấy vị trí bằng cách khác (ví dụ một biến lưu lần bấm gần nhất) thì có gì rủi ro?

Cách lưu biến "lần bấm gần nhất" chỉ đúng khi mọi thao tác đều đi qua đúng một đường. Trong bài có ít nhất bốn đường có thể đổi dòng đang chọn: chạm để xem chi tiết, giữ lâu để mở context menu, bấm nút + thêm món, và lọc/sắp xếp làm danh sách đổi thứ tự. Chỉ cần một đường quên cập nhật biến là context menu thao tác trên dòng cũ — người dùng bấm Xóa ở dòng 3 nhưng mất dòng 5.

Nguy hiểm hơn là khi danh sách đang lọc: vị trí hiển thị (0, 1, 2…) không trùng vị trí trong dữ liệu gốc. Bài xử lý bằng cách lấy đối tượng MonAn từ hienThi.get(info.position) rồi mới xóa theo danhSach.indexOf(mon). AdapterContextMenuInfo do Android gửi kèm chính là nguồn tin cậy duy nhất cho câu hỏi "vừa giữ lâu dòng nào", vì nó được tạo ngay tại thời điểm sự kiện.

2. Ở kịch bản 7, nếu dùng danhSach.add(daXoa) thay vì danhSach.add(viTri, daXoa) thì chuyện gì xảy ra?

Món vẫn quay lại nhưng nằm ở cuối danh sách, không phải vị trí thứ 3 như trước khi xóa. Người dùng thấy "hoàn tác" mà thực đơn đã bị đổi thứ tự — với danh sách 5 món thì khó nhận ra, với 50 món thì món vừa cứu rơi xuống tận đáy và gần như không tìm lại được.

Trong bài còn một hệ quả thứ hai: nếu đang sắp xếp theo giá, chèn vào cuối làm danh sách mất thứ tự tăng dần mà không có thao tác sắp xếp nào của người dùng. Vì vậy phải giữ lại cả đối tượng lẫn vị trí trước khi xóa (final MonAn daXoa = danhSach.remove(viTri)) và chèn lại đúng chỗ bằng add(viTri, daXoa).

3. Ở kịch bản 8, hãy mô tả bằng lời vì sao phải gán lại listener sau show().

AlertDialog gắn listener của setPositiveButton vào nút và tự gọi dismiss() sau khi callback chạy xong. Đó là hành vi mặc định, không có tham số nào để tắt. Vì vậy nếu kiểm tra dữ liệu bên trong callback đó, hộp thoại đã đóng trước khi ta kịp return — người dùng mất toàn bộ nội dung vừa gõ chỉ vì thiếu một ô.

Gán lại OnClickListener sau show() thay thế listener mặc định bằng listener của ta. Lúc này AlertDialog không còn tự đóng nữa; code gọi dialog.dismiss() đúng một lần, sau khi cả ba điều kiện đã hợp lệ. getButton(...) phải gọi sau show() vì trước đó các nút chưa được dựng.

Phần 8. Bài nâng cao (chọn 3 trong 4 bài)

Bài làm chọn NC1, NC2 và NC3. Không chọn NC4 (thay context menu bằng BottomSheetDialog) vì context menu chính là yêu cầu bắt buộc trị giá 2,0 điểm của đề; thay nó đi là đánh đổi điểm bắt buộc lấy điểm nâng cao. Ba bài còn lại đều làm tăng trải nghiệm mà không phá cấu trúc menu – context menu – dialog của bài.

8.1. NC1 — SearchView lọc danh sách ngay khi gõ

SearchView khai báo trong menu với actionViewClass và showAsAction="always|collapseActionView": khi chưa dùng nó chỉ là chiếc kính lúp, chạm vào mới mở rộng ra ô nhập. Listener được gắn trong onPrepareOptionsMenu:

```
itemTimKiem = menu.findItem(R.id.action_tim_kiem);
searchView = (SearchView) itemTimKiem.getActionView();
searchView.setQueryHint(getString(R.string.hint_tim));
searchView.setOnQueryTextListener(new SearchView.OnQueryTextListener() {
    @Override public boolean onQueryTextSubmit(String query) {
        searchView.clearFocus();   // ẩn bàn phím, giữ kết quả
        return true;
    }
    @Override public boolean onQueryTextChange(String newText) {
        tuKhoa = newText.trim();
        locVaCapNhat();            // lọc ngay khi đang gõ
        return true;
    }
});
```

Bộ lọc dùng MonAn.khongDau() để bỏ dấu tiếng Việt trước khi so sánh, nhờ đó gõ “bun” vẫn tìm ra “Bún chả Hà Nội” dù bàn phím không có dấu. Khi đóng ô tìm kiếm, onMenuItemActionCollapse xóa từ khóa và trả danh sách về đủ 5 món. Chip thống kê đổi thành “Hiện 1/5 món · 190.000 đ” để người dùng biết đang xem một phần của thực đơn — xem Hình 3c.

8.2. NC2 — hộp thoại chọn mức giá bằng setSingleChoiceItems

Bốn mức giá khai báo trong res/values/strings.xml dưới dạng string-array, nên không có chuỗi tiếng Việt nào nằm trong code Java:

```
<string-array name="muc_gia">
    <item>Tất cả mức giá</item>
    <item>Dưới 30.000 đ</item>
    <item>Từ 30.000 đ đến 45.000 đ</item>
    <item>Trên 45.000 đ</item>
</string-array>

private void hienDialogChonMucGia() {
    final String[] muc = getResources().getStringArray(R.array.muc_gia);
    final int[] luaChon = {mucGia};   // biến tạm: chỉ áp dụng khi bấm Áp dụng

    new MaterialAlertDialogBuilder(this)
            .setTitle(R.string.chon_muc_gia)
            .setSingleChoiceItems(muc, mucGia, (d, which) -> luaChon[0] = which)
            .setPositiveButton(R.string.ap_dung, (d, w) -> {
                mucGia = luaChon[0];
                locVaCapNhat();
            })
            .setNegativeButton(R.string.huy, null)   // Hủy = giữ mức giá cũ
            .show();
}
```

Điểm cần chú ý là biến tạm luaChon: chọn radio chỉ đổi lựa chọn đang chờ, chưa đụng tới mucGia. Chỉ khi bấm Áp dụng thì mức giá mới được gán và danh sách mới lọc lại; bấm Hủy thì mọi thứ giữ nguyên — đúng tinh thần “Positive mới áp dụng” của đề. Bộ lọc giá kết hợp được với từ khóa NC1: locVaCapNhat kiểm tra cả hai điều kiện. Hình 3d và Hình 4 cho thấy chọn “Trên

45.000 đ” rồi Áp dụng chỉ còn “Bún chả Hà Nội” 50.000 đ, chip hiện “Hiện 1/5 món · 190.000 đ”.



Hình 4. NC2: sau khi Áp dụng “Trên 45.000 đ”, danh sách chỉ còn món 50.000 đ.

8.3. NC3 — MaterialAlertDialogBuilder thay AlertDialog.Builder

Toàn bộ bốn loại hộp thoại của bài (chi tiết, xác nhận xóa, tự thiết kế, chọn mức giá) đều dùng MaterialAlertDialogBuilder. API giống hệt AlertDialog.Builder nên chỉ cần đổi tên lớp, nhưng giao diện theo Material 3 và tự lấy màu từ theme.

Tiêu chí	AlertDialog.Builder	MaterialAlertDialogBuilder
Bo góc / nền	Theo theme nền tảng, thường vuông hơn	Bo góc lớn, nền tonal theo Material 3
Màu nút	Màu accent của theme cũ	colorPrimary của theme Material 3
Tiêu đề	Kiểu text thường	Kiểu headline của Material 3
Phụ thuộc	androidx.appcompat	com.google.android.material + theme Material3
Cách dùng	new AlertDialog.Builder(this)	new MaterialAlertDialogBuilder(this) — cùng API

Trong ảnh chụp có thể thấy rõ khác biệt: các hộp thoại của bài có nền hồng nhạt lấy từ colorSurface của theme, bo góc lớn và nút chữ màu nâu đậm (colorPrimary). Đây là lý do bài chọn NC3 dù nó "chỉ đổi một dòng": nó làm cả bốn hộp thoại đồng bộ với MaterialToolbar và ListView, thay vì mỗi chỗ một kiểu.

Phần 9. Thang điểm và tự đánh giá

Tiêu chí	Điểm	Minh chứng trong bài
Options menu đủ 4 mục, hoạt động đúng	2,0	Mục 2.2–2.3; Hình 1b, 1c; log "Đã sắp xếp theo giá"
Context menu Sửa / Xóa, lấy đúng vị trí dòng	2,0	Mục 3; Hình 1d, 2a; log "vị trí 2"
Hộp thoại xác nhận trước khi xóa, lời hỏi rõ ràng	1,5	Mục 4; Hình 2a, 2b
Dialog tự thiết kế, không đóng khi dữ liệu sai	1,5	Mục 5; Hình 3a, 3b
Snackbar Hoàn tác trả về đúng vị trí cũ	1,0	Mục 6; Hình 2c, 2d; log "về vị trí 2"
Bảng 8 kịch bản có ảnh minh chứng	1,0	Mục 7, Hình 1–3
Bài nâng cao (đề yêu cầu ≥ 2 bài)	1,0	NC1 (8.1) + NC2 (8.2) + NC3 (8.3)
Tổng	10	

Phần 10. Xử lý sự cố thường gặp

Hiện tượng	Nguyên nhân	Cách xử lý
Menu không hiện trên thanh tiêu đề	Quên return true, hoặc theme là NoActionBar	Thêm MaterialToolbar + setSupportActionBar (bài dùng cách này)
Giữ lâu không ra context menu	Chưa registerForContextMenu, hoặc listener A9 cũ nuốt sự kiện	Đăng ký context menu và xóa hẳn setOnItemLongClickListener
Context menu thao tác nhầm dòng khi đang lọc	Lấy vị trí từ biến lưu lần bấm gần nhất	Lấy từ AdapterContextMenuInfo.position, rồi xóa theo danhSach.indexOf(mon)
Dialog đóng mất dù dữ liệu sai	Gán listener ngay trong setPositiveButton	Gán null lúc tạo, gán lại listener sau show()
Hoàn tác xong món nhảy xuống cuối	Dùng add(daXoa) thay vì add(viTri, daXoa)	Giữ lại vị trí trước khi xóa
Không tìm thấy lớp Snackbar	Thiếu com.google.android.material	Thêm dependency material rồi Sync Now
Context menu không có tiêu đề	Quên menu.setHeaderTitle(...)	Đặt tiêu đề bằng tên món trong onCreateContextMenu
Kính lúp không mở rộng thành ô nhập	Thiếu setSupportActionBar hoặc thiếu collapseActionView	Đặt showAsAction="always|collapseActionView" và kiểm tra toolbar
Dialog Sửa mở ra nhưng ô trống	Truyền null thay vì đối tượng món	Lấy mon từ hienThi.get(info.position) rồi truyền vào hienDialogThemSua(mon)
Ảnh chụp bị bấm lệch chỗ trên emulator	Toạ độ suy từ ảnh hiển thị bị co tỉ lệ	Lấy bounds thật từ uiautomator dump trước khi bấm

Phần 11. Câu hỏi ôn tập

1. Khi nào dùng Toast, khi nào Snackbar, khi nào AlertDialog?

Toast khi việc đã xong và không cần phản hồi (đã sắp xếp, đã lưu). Snackbar khi việc đã xong nhưng người dùng có thể muốn sửa (đã xóa — Hoàn tác). AlertDialog khi phải hỏi trước một việc nguy hiểm hoặc cần người dùng quyết định rõ ràng (xóa, gửi, thanh toán).

2. Làm sao biết người dùng vừa giữ lâu dòng nào?

Android gửi kèm AdapterView.AdapterContextMenuInfo trong MenuItem; đọc info.position là chỉ số dòng được giữ lâu. Trong bài, chỉ số đó thuộc hienThi nên lấy MonAn mon = hienThi.get(info.position) rồi thao tác trên đối tượng, tránh nhầm khi danh sách đang lọc.

3. Vì sao phải gán lại listener sau khi gọi show()?

Vì listener đặt qua setPositiveButton bị AlertDialog tự động đóng hộp thoại sau khi chạy. Gán lại sau show() (lúc các nút đã tồn tại) cho phép code kiểm tra dữ liệu và chỉ gọi dismiss() khi hợp lệ.

4. Hoàn tác đúng cách cần giữ lại những gì?

Giữ cả đối tượng vừa xóa lẫn vị trí của nó trong danh sách gốc. Thiếu đối tượng thì không chèn lại được; thiếu vị trí thì món rơi xuống cuối. Với xóa tất cả, giữ cả bản sao danh sách.

5. Vì sao dòng return super.onOptionsItemSelected(item) ở cuối lại quan trọng?

Nó chuyển những mục mình không xử lý về cho lớp cha (ví dụ nút Back trên thanh tiêu đề, phím menu phần cứng). Bỏ dòng này thì các hành vi mặc định đó im lặng không chạy — lỗi không crash nên rất khó tìm.

Phần 12. Kết luận và sản phẩm nộp

Bài lab đã hoàn thành đủ yêu cầu: options menu 4 mục nạp từ XML và xử lý bằng if–else; context menu lấy đúng vị trí dòng qua AdapterContextMenuInfo; hộp thoại xác nhận nêu rõ tên món; dialog tự thiết kế không đóng khi dữ liệu sai; Snackbar Hoàn tác trả món về đúng vị trí cũ; NC1–NC3 đều chạy trên thiết bị thật.

- Repo GitHub (public): https://github.com/nvquangggit-sys/A11_231A290134 — có README, CREDITS và lịch sử commit.
- Minh chứng trong repo: docs/screenshots (13 ảnh) và docs/logs/logcat_a11.txt.
- Video demo: docs/A11_231A290134_demo.mp4, dài 1 phút 02 giây — Sắp xếp → giữ lâu Sửa → giữ lâu Xóa → Hủy → Xóa → Hoàn tác.
- Báo cáo: tệp này (DOCX) và bản PDF cùng tên.
- Nguồn ảnh: Wikimedia Commons, CC BY-SA 3.0/4.0 — xem CREDITS.md.

Bài học lớn nhất: menu chỉ là giao diện; phần khó là biết chính xác người dùng đang thao tác trên đối tượng nào và giữ cho thao tác đó có thể cứu được — sang Lab A12 với SQLite, hai câu hỏi ấy vẫn y nguyên.